Index

Index **1**
SECTION - 1 **3**
1. What is tracing? **3**
Difference between tracing and logging 3
2. ftrace **4**
Why do you need it? 4
What can we do using ftrace? 4
3. Ftrace **5**
Minimum kernel configuration required for ftrace 5
How to check whether ftrace is already enabled or not? 5
4. Important files in tracefs **6**
available_tracers 6
current_tracer 6
5. How to enable a tracer? **7**
Viewing the trace 7
6. Function Graph Tracer **8**
Important Points 8
7. Function Filtering **9**
What functions are available for the filter? 9
Limit What Functions you trace 9
Example: 9
8. Wild cards **11**
9. Q/A **12**
10. ftrace on simple kernel module **13**
11. Starting and Stopping the trace **15**
12. trace vs trace_pipe **16**
13. Filtering function graph tracing **17**
What's the benefit 17
14. Tracing a specific Process **18**
15. Trace all functions related to a specific module **20**
16. trace_printk **21**
With trace_printk 21
17. Finding kernel functions taking > 10ms **22**
18. Finding out how user space code gets into kernel **23**
SECTION - 2 **24**
1. Flags(irqs-off) **24**
irqs-off: 24
2. Flags(need-resched) **25**
3. Flags (hardirq/softirq) **27**
4. Flags(preempt-depth) **29**
5. Start tracing on a particular function **31**

6. trace options 32
7. trace options 33
8. Disabling interrupts info in function graph 35
9. Who calls this kernel function 36
10. trace_marker 37
11. Kernel function profiling 40
12. Finding out maximum time interrupts are disabled 41
13. Finding latencies at kernel startup 42
14. Find deepest kernel stack 43
15. Capturing Ftrace to oops when kernel panic 44
16. snapshot 45
SECTION - 3 46
1. Tracing Events 46
2. Tracing Events (available_events) 51
3. Format file 53
4. Filtering Events 56
5. Setting filters - More examples 58
6. Subsystem filters 59
7. Event Triggers 60
8. Histogram 63
9. Event Tracing via the 'set_event' interface 65
10. PID filtering 66
11. Tracing Event at Boot 67

SECTION - 1

1. What is tracing?

A traditional debugger allows you to inspect the system state once the system is halted
i.e., after an error has been detected, but doesn't reveal the events leading to the error.

To understand why an event took place, the relevant context has to be restored. This requires tracing

Tracing is the process of collecting information on the activity in a working system

With tracing, program execution is recorded during run-time, allowing for later analysis of the trace

Tracing provides developers with information useful for debugging.

Difference between tracing and logging

Tracing is sometimes compared to logging. Just like logs, tracing data can be read as it is.

With tracing, information is written about low-level events.
These numbers in the hundreds or even thousands.

With logging, information is written about higher-level events, which are much less frequent
These includes users logging into the system, application errors, database transaction etc

2. ftrace

Official Tracer of Linux Kernel. It let's you trace Linux Kernel function calls

Added in 2.6.31 (2009)

Mostly developed by Steven Rostedt

Ftrace is named after the function tracer it also includes many more functionalities

Why do you need it?

Say you write a device driver, debug it, and confirm that everything's fine.

Then you run it inside the kernel and find that it's not working as you expected, or something else breaks when you run it.

With help of ftrace, you can find out what functions are being called at kernel level and easily debug the issue

What can we do using ftrace?

Debugging Linux Kernel
Analyzing Latencies in Linux Kernel
Learn and observe the flow of Linux Kernel
Trace context switches
Length of the time the interrupts are disabled

3. Ftrace

Enabled in most distros by default

Activated by Kernel Hacking -> Tracers

Minimum kernel configuration required for ftrace

```
CONFIG_FTRACE --> "Tracers"
CONFIG_FUNCTION_TRACER --> Kernel Function Tracer
CONFIG_FUNCTION_GRAPH_TRACER --> Kernel Function Graph Tracer
CONFIG_STACK_TRACER --> Traces Max stack
```

How to check whether ftrace is already enabled or not?

Most of the Linux distributions already provide ftrace, to check verify "tracing" directory exists or not.

```
# ls /sys/kernel/tracing
```

mount the tracefs by any of the two steps:

1. Adding an entry into /etc/fstab:

```
tracefs       /sys/kernel/tracing       tracefs defaults        0       0
```

2. Using the mount command:

```
#mount -t tracefs nodev /sys/kernel/tracing
```

After mounting tracefs, you will have access to the control and output files of ftrace.

It was initially in debugfs(/sys/kernel/debug/tracing)

4. Important files in tracefs

All manipulations are done with simple files operations (echo/cat) in tracefs

ftrace is built around smart lockless ring buffer implementation.

This buffer stores all the tracing information and is exposed as a file in tracefs

available_tracers

Lists which tracers are configured

Important ones are:

1. **nop:** No Action (No Tracer Enabled)
2. **function:** Trace Kernel Functions Entry
3. **function_graph:** Trace Kernel Functions Entry and Exit thus allowing to build a call graph
4. **blk:** Block Tracer/blktrace
5. **mmiotrace:** Trace interactions between drivers and hardware

Default tracer is nop

current_tracer

The tracing program presently running

5. How to enable a tracer?

```
$ echo 'name of tracer' > current_tracer

$ echo 'function' > current_tracer

$ echo 'function_graph' > current_tracer

$ echo 'nop' > current_tracer
```

Viewing the trace

'trace' is the file where tracing data is saved in human-readable format

```
$ cat trace
```

Those are functions that are happening on the computer right now

You can see from the cat output:

First line specifies the tracer installed, in our case "function"

Next each line prints the following information:

- Process Name
- PID
- CPU that the trace executed on
- Timestamp in seconds with the decimal places to microseconds. This timestamp is time since boot
- Function being traced
- Parent that call the function

6. Function Graph Tracer

Function Graph Tracer is built on top of Function Tracer.
It also records the return address. By this you will come to know when a function enters and function exits.

```
$ echo function_graph > current_tracer
```

Function_graph tracer
- tracks the entry of the function
- tracks the exit of the function
- Execution Time
- CPU on which it is running

Important Points

Start of the function is denoted with '{' and end of the function is denoted with '}'.
Functions that do not call any other functions, simply end with ';', also called as leaf functions
Duration column reports the time spent in the corresponding function. These numbers are only for the leaf functions, and the '}' symbol.

When the duration is greater than 10 microseconds, a '+' is shown in the DURATION column
When the duration is greater than 100 microseconds, a '-' is shown in DURATION column
When the duration is greater than 1000 microseconds, a '#' is shown in DURATION column
When the duration is greater than 10 milliseconds, a '*' is shown in DURATION column
When the duration is greater than 100 milliseconds, a '@' is shown in DURATION column
When the duration is greater than 1 seconds, a '$' is shown in DURATION column

7. Function Filtering

The ftrace printout can be big, and finding exactly what it is you're looking for can be extremely difficult.

We can use filters to simplify our search.

The printout will only display information about functions we're interested in.

What functions are available for the filter?

All the functions that can be traced are present in available_filter_functions

Limit What Functions you trace

If you want only enable few of the functions or disable few of the functions, then you can use the below files

set_ftrace_filter - Only trace these functions
set_ftrace_notrace - Do not trace these functions
(notrace takes precedence over filter)

Just echo function names into the files

```
echo function_name > set_ftrace_filter
```

Can add multiple (white space delimited)

```
echo function1 function2 > set_ftrace_filter
```

Append with the bash concatenation ">>"

```
echo function >> set_ftrace_filter
```

Clear with just writing nothing to it

```
echo > set_ftrace_notrace
```

Example:

```
echo function > current_tracer
echo do_page_fault > set_ftrace_filter
cat trace

echo > set_ftrace_filter
echo kfree > set_ftrace_notrace
```

```
echo vfs_* > /sys/kernel/debug/tracing/set_ftrace_filter
echo schedule > set_ftrace_filter
```

set_ftrace_filter and set_ftrace_notrace affect function_graph as well

```
echo do_IRQ > set_ftrace_filter
echo function_graph > current_tracer
cat trace
```

This shows you the interrupt times, interrupt latencies of all the interrupts that are happening in your system

8. Wild cards

It supports minor wild cards "*" and "?"

value* - Will select all functions that start with string "value"
*value - Will select all functions that end with string "value"
value - Will select all functions that start and end with string "value"

```
echo 'xen*' > set_ftrace_filter

echo '*lock*' > set_ftrace_filter

echo '*mutex*' > set_ftrace_filter
```

Extended glob matches (Started in 4.10)

```
echo '?raw_*lock' > set_ftrace_filter
```

9. Q/A

Question: Does available_filter_functions file show only exported symbols or all kernel symbols?

What happens if there are two functions with same function name?

Answer: Does available_filter_functions file show only exported symbols or all kernel symbols?
Answer: Every function which is present in kallsysms, which is basically every single function

What happens if there are two functions with same function name?
Answer: It will enable both of them.

10. ftrace on simple kernel module

Github link - https://github.com/kd1995-max/Ftrace-practice/tree/main/section-1/10

Steps (Short & Clear)

1. Clone the repo

```
git clone https://github.com/kd1995-max/Ftrace-practice
cd <repo_folder>
```

2. Build the kernel module

```
make
```

Generates `hello.ko`

3. Build user application

```
gcc -o userapp userapp.c
```

4. Load the module

```
sudo insmod hello.ko
```

5. Check:

```
dmesg | tail
```

6. Check device node

```
ls -l /dev/mychardev
```

7. Run user app

```
./userapp
```

Verify driver calls:

```
dmesg | tail
```

8. Trace using ftrace

```
cd /sys/kernel/tracing
echo function > current_tracer
echo device_open > set_ftrace_filter
echo device_read >> set_ftrace_filter
echo device_write >> set_ftrace_filter
echo device_release >> set_ftrace_filter
echo 1 > tracing_on
```

Run app again:

```
./userapp
cat trace
```

9. Measure time

```
echo function_graph > current_tracer
./userapp
cat trace
```

10. Remove module

sudo rmmod hello

11. Starting and Stopping the trace

Suppose you only want to trace what is happening when you run a specific test, you can use tracing_on file

The file tracing_on is used to disable the ring buffer from recording data

Its a file, on/off switch. To make the ring buffer writeable or not

```
echo 0 > tracing_on
```

Everything else still happens with the tracers and they will still incur most of their overhead.

They do notice that the ring buffer is not recording and will not attempt to write any data, but the calls that the tracers make are still performed.

To re-enable the ring buffer, simply write a '1' into that file:

```
echo 1 > tracing_on
```

A common run might be:

```
echo 0 > tracing_on
echo function_graph > current_tracer
echo 1 > tracing_on; run_test; echo 0 > tracing_on
```

The first line disables the ring buffer from recording any data.

The next enables the function graph tracer.

The overhead of the function graph tracer is still present but nothing will be recorded into the trace buffer. The last line enables the ring buffer, runs the test program, then disables the ring buffer.

This narrows the data stored by the function graph tracer to include mostly just the data accumulated by the run_test program.

.

12. trace vs trace_pipe

ftrace internally uses a ring buffer of fixed size.

The size can be find out by reading "buffer_size_kb" file.

If the buffer becomes full, it will overwrite the starting tracing entries.

```
cat buffer_size_kb
1410
```

To read this ring buffer, we can cat either of the two files present in /sys/kernel/tracing folder:
trace
trace_pipe

trace is a non-consuming read. It pauses the tracing when you are reading
trace_pipe is consumer. It doesn't stop the tracing

Each time you read the contents of "trace" file it will print the same tracing entries, whereas with trace_pipe, each time you read the file the data is consumed, and in the next read data will be different also if there is no data trace_pipe will block.

```
echo 0 > tracing_on
cat trace
cat trace
cat trace_pipe
cat trace_pipe
```

13. Filtering function graph tracing

set_graph_function:

Similar to set_ftrace_filter
Functions listed in this file will cause the function graph tracer to only trace these functions and the functions that they call

What's the benefit

Useful to find out what calls made by a kernel function

Eg.

```
echo vfs_read > set_graph_function
echo function_graph > current_tracer
```

set_graph_notrace

Disable function graph tracing when the function is hit until it exits the function.

14. Tracing a specific Process

With ftrace, we can trace kernel calls only related to a specific process by echoing pid in the set_ftrace_pid file.

Neat little trick to trace only, what you want

```
echo 0 > tracing_on
echo function > current_tracer
sh -c 'echo $$ > set_ftrace_pid; echo 1 > tracing_on; exec myprog;'
```

Example

```
sh -c 'echo $$ > set_ftrace_pid; echo 1 > tracing_on; exec echo hello;'
```

traceme.sh script

```
#!/bin/bash

DEBUGFS=`grep debugfs /proc/mounts | awk '{print $2;}'`
echo nop > $DEBUGFS/tracing/current_tracer
echo > $DEBUGFS/tracing/trace
echo $1
echo $$ > $DEBUGFS/tracing/set_ftrace_pid
echo function > $DEBUGFS/tracing/current_tracer
echo 1 > $DEBUGFS/tracing/tracing_on
exec $*
echo 0 > $DEBUGFS/tracing/tracing_on
```

```
traceme.sh ls
cat /sys/kernel/debug/tracing/trace | less
cat /sys/kernel/debug/tracing/trace > output
```

traceme_with_filter.sh

```
#!/bin/bash

DEBUGFS=`grep debugfs /proc/mounts | awk '{print $2;}'`
echo nop > $DEBUGFS/tracing/current_tracer
echo > $DEBUGFS/tracing/trace
echo $1
echo $$ > $DEBUGFS/tracing/set_ftrace_pid
echo function > $DEBUGFS/tracing/current_tracer
echo vfs_* > /sys/kernel/debug/tracing/set_ftrace_filter
echo 1 > $DEBUGFS/tracing/tracing_on
exec $*
echo 0 > $DEBUGFS/tracing/tracing_on
```

15. Trace all functions related to a specific module

To trace all functions of a specific module, for example e1000 (Ethernet driver) you can pass the following to set_ftrace_filter.

```
echo ':mod:e1000' > set_ftrace_filter
cat set_ftrace_filter | head -n 10

ifconfig enp0s3 up
ifconfig enp0s3 down
```

traceme.sh

```
#!/bin/bash

DEBUGFS=`grep debugfs /proc/mounts | awk '{print $2;}'`
echo 0 > $DEBUGFS/tracing/tracing_on
echo nop > $DEBUGFS/tracing/current_tracer
echo > $DEBUGFS/tracing/trace
echo  > $DEBUGFS/tracing/set_ftrace_pid
echo function > $DEBUGFS/tracing/current_tracer
echo ':mod:e1000' > /sys/kernel/debug/tracing/set_ftrace_filter
echo 1 > $DEBUGFS/tracing/tracing_on
read enter
echo 0 > $DEBUGFS/tracing/tracing_on
```

16. trace_printk

printk() is the king of all debuggers, but it has a problem.

Using printk in interrupt context such as timer interrupts, scheduler, network can create a live lock

Sometimes bug disappear when we add few printk's if something is time sensitive

printk when writing to the serial console may take several milliseconds

With trace_printk

writing will be in the order of microseconds as it writes to a ring buffer instead of console

can be used in any context (interrupt, scheduler, NMI Code)

can be read via the 'trace' file

The performance advantage of trace_printk() lets you record the most sensitive areas of the kernel with very little impact.

For example you can add something like this to the kernel or module:

```
trace_printk("Hello %s\n", "LWL");
```

Note: trace_printk() output will appear in any tracer, even the function and function graph tracers.

17. Finding kernel functions taking > 10ms

ftrace uses a 'tracing_thresh' file and compares the function duration if greater than the value present in the file, it logs tracing entries in the ring buffer.

By default the value is zero, it means ftrace doesn't care about this. The value written in 'tracing_thresh' is in microseconds.

So, if you want to find out the kernel functions which are taking more than 10ms , you need to write

```
echo '10000' > tracing_thresh
```

Set back zero in tracing_thresh, if you want to trace all the functions.

```
echo '0' > tracing_thresh
```

18. Finding out how user space code gets into kernel

We can find out what is the first kernel function called when the user space code gets into the kernel using 'max_graph_depth' file

Note: This can be used only with function_graph tracer

max_graph_depth file contains the maximum depth it will trace into a function

Default value: 0 - This means unlimited

If you 'echo 1 > max_graph_depth', it will only trace the first function and ignore the other functions.

```
cat trace
```

'do_syscall_64()' is the function through which the user space code gets into the kernel space.

If we want to find out the next function after 'do_syscall_64()':

```
echo 2 > max_graph_depth'

echo 0 > tracing_on

echo 1 > max_graph_depth

echo function_graph > current_tracer

sh -c 'echo $$ > set_ftrace_pid; echo 1 > tracing_on; exec echo hello'

cat trace
```

SECTION - 2

1. Flags(irqs-off)

```
# tracer: nop
#
# entries-in-buffer/entries-written: 79/79   #P:6
#
#                                _-----=> irqs-off
#                               / _----=> need-resched
#                              | / _---=> hardirq/softirq
#                              || / _--=> preempt-depth
#                              ||| /     delay
```

irqs-off:

irqs-off means **CPU interrupts are disabled** at that moment.

Why it matters:

- When interrupts are off, the CPU **cannot respond to hardware interrupts**
- Used in **critical sections** to protect shared data

`irqs-off = 1` → Interrupts are **disabled**
`irqs-off = .` → Interrupts are **enabled**

Github link - https://github.com/kd1995-max/Ftrace-practice/tree/main/section-2/1

2. Flags(need-resched)

```
# tracer: nop
#
# entries-in-buffer/entries-written: 79/79   #P:6
#
#                                _-----=> irqs-off
#                               / _----=> need-resched
#                              | / _---=> hardirq/softirq
#                              || / _--=> preempt-depth
#                              ||| /     delay
```

need-resched:

'**N**' both TIF_NEED_RESCHED and PREEMPT_NEED_RESCHED is set,
'**n**' only TIF_NEED_RESCHED is set,
'**p**' only PREEMPT_NEED_RESCHED is set,
'**.**' otherwise.

Preemption

Preemption is the process of forcefully grabbing CPU from a user or kernel context and giving it to someone else

How does it work?

A timer interrupt (also called tick) interrupts the task/process that is running and makes a decision whether a task running in user space or a kernel code path (executing on behalf of syscall) is to be preempted

How is the decision made?

This decision is based on whether
the task has been running long-enough and something high priority woke up now and needs CPU

Kernel function which handles this is scheduler_tick()

In this function, decision is made that the process ran long enough based on its runtime

If yes, sets the TIF_NEED_RESCHED for the current task

Even though this flag is set at this point, the task is not going to be preempted yet.
This is because preemption happens at specific points such as exit of interrupts

PREEMPT_NEED_RESCHED

In x86 architectures a PREEMPT_NEED_RESCHED bit is also used in preempt_count, for optimization of deciding to reschedule.

3. Flags (hardirq/softirq)

```
# tracer: nop
#
# entries-in-buffer/entries-written: 79/79   #P:6
#
#                                _-----=> irqs-off
#                               / _----=> need-resched
#                              | / _---=> hardirq/softirq
#                              || / _--=> preempt-depth
#                              ||| /     delay
```

hardirq/softirq:

'Z' - NMI occurred inside a hardirq
'z' - NMI is running
'H' - hard irq occurred inside a softirq.
'h' - hard irq is running
's' - soft irq is running
'.' - normal context.

Hard IRQ

- **Immediate hardware interrupt**
- Runs **directly in interrupt context**
- Must be **very fast** (no sleeping)

Soft IRQ

- **Deferred interrupt work**
- Runs **after hard IRQ**
- Used for heavier processing (e.g. network RX)

NMI (Non-Maskable Interrupt)

- **Highest-priority interrupt**
- **Cannot be disabled**
- Used for **critical hardware errors** (watchdog, HW failure)

`hardirq/softirq` column meanings

- `Z` → **NMI occurred inside a hard IRQ**

  While the CPU was already handling a hard interrupt, a Non-Maskable Interrupt (NMI) arrived and preempted it.

- `z` → **NMI is currently running**
- `H` → **Hard IRQ occurred inside a soft IRQ**

  While the CPU was executing a soft IRQ, a hardware interrupt (hard IRQ) arrived and preempted it.

- `h` → **Hard IRQ is running**
- `s` → **Soft IRQ is running**
- `.` → **Normal process context**

Github link : https://github.com/kd1995-max/Ftrace-practice/tree/main/section-⅔

4. Flags(preempt-depth)

```
# tracer: nop
#
# entries-in-buffer/entries-written: 79/79   #P:6
#
#                                _-----=> irqs-off
#                               / _----=> need-resched
#                              | / _---=> hardirq/softirq
#                              || / _--=> preempt-depth
#                              ||| /     delay
```

What is `preempt-depth` (in ftrace)?

`preempt-depth` shows how many times kernel preemption is currently disabled.

In simple words:
It tells **"Can the current task be stopped and switched out right now?"**

What is kernel preemption?

- **Preemption = CPU can stop current task and run another**
- Improves **responsiveness**

How preemption is disabled

- Kernel disables preemption when it must **not be interrupted**
- Example: while holding a **spinlock**

Internally:

- Kernel increases a counter called **`preempt_count`**
- If `preempt_count > 0` → **no preemption allowed**

This counter value is shown as **`preempt-depth`** in ftrace.

Simple example

preempt-depth = 0 → task can be preempted
preempt-depth = 1 → preemption disabled once
preempt-depth = 2 → nested disable (spinlock inside spinlock)

How kernel controls it

```
preempt_disable();   // preempt-depth +1
preempt_enable();    // preempt-depth -1
```

- Calls are **nestable**
- Preemption is enabled only when depth returns to **0**

Why spinlocks disable preemption

- To protect **shared data**
- If task switches while holding a lock → **deadlock**

So:

> **Any time a spinlock is held, preempt-depth > 0**

When kernel checks this

- On **return from system call**
- On **return from interrupt**

If `preempt-depth != 0` → **do not reschedule**

Kernel config options

CONFIG_PREEMPT

- **Full kernel preemption**
- Lowest latency, more context switches

`CONFIG_PREEMPT_VOLUNTARY` (default)

- Kernel allows preemption only at **safe points**
- Balanced **latency + throughput**

Preempt OFF

- Kernel never preempts itself
- Best throughput, worst latency

Github: https://github.com/kd1995-max/Ftrace-practice/tree/main/section-2/4

5. Start tracing on a particular function

We know the 'tracing_on' file is used to disable/enable writing the traces on the ring buffer.

```
echo '0' > tracing_on //to disable tracing
echo '1' > tracing_on // to enable tracing
```

ftrace also provides functionality to start tracing on a particular function call execution or stop tracing on a particular function execution.

Syntax: function:command[:count]

The command will be executed on the start of the function. The value of command can be

"traceon" to start the trace
"traceoff" to stop the trace
"stacktrace"

The count is optional, specifies the number of times the command should be executed.
If there is no count specified, the command will be executed each time the function is called

```
echo 'nop' > current_tracer
echo 'ksys_read:traceoff:1' > set_ftrace_filter
cat set_ftrace_filter
echo 'function' > current_tracer
```

How to remove this?

Using '!' operator

```
echo '!ksys_read:traceoff' > set_ftrace_filter
```

6. trace options

Modifies the way tracers may work

Modifies output format

Some tracers have their own options

Two ways to modify the options

file trace_options

shows only global or current tracer options

to enable `echo option_name > trace_options`

to disable `echo nooption_name > trace_options`

```
cat trace_options
```

There are 27 options available for the user to control

The words which begin with "no" represent trace options that are disabled.

Let's try few options

1. sym-offset: Enabling this option, you will see function name plus offset in the function. By default, this is disabled.

2. print-parent: This option displays the calling function. By default, this option is disabled.

3. function-fork: This option enables tracing the children when set_ftrace_pid is set.

4. irq-info: Displays the interrupt, preempt count, need resched data information in the trace

5. context-info: Disabling this option, will hide the Task-PID, CPU, Timestamp and other useful information.

7. trace options

options directory

shows global options

shows current tracer options

starting with v4.4 - shows all tracer options

to enable `echo 1 > options/option_name`

to disable `echo 0 > options/option_name`

```
root@RPI-3:/home/rpi-3# ls -ltr /sys/kernel/tracing/options/
total 0
-rw-r----- 1 root root 0 Jan  1  1970 verbose
-rw-r----- 1 root root 0 Jan  1  1970 userstacktrace
-rw-r----- 1 root root 0 Jan  1  1970 trace_printk_dest
-rw-r----- 1 root root 0 Jan  1  1970 trace_printk
-rw-r----- 1 root root 0 Jan  1  1970 sym-userobj
-rw-r----- 1 root root 0 Jan  1  1970 sym-offset
-rw-r----- 1 root root 0 Jan  1  1970 sym-addr
-rw-r----- 1 root root 0 Jan  1  1970 stacktrace
-rw-r----- 1 root root 0 Jan  1  1970 record-tgid
-rw-r----- 1 root root 0 Jan  1  1970 record-cmd
-rw-r----- 1 root root 0 Jan  1  1970 raw
-rw-r----- 1 root root 0 Jan  1  1970 print-parent
-rw-r----- 1 root root 0 Jan  1  1970 printk-msg-only
-rw-r----- 1 root root 0 Jan  1  1970 pause-on-trace
-rw-r----- 1 root root 0 Jan  1  1970 overwrite
-rw-r----- 1 root root 0 Jan  1  1970 markers
-rw-r----- 1 root root 0 Jan  1  1970 latency-format
-rw-r----- 1 root root 0 Jan  1  1970 irq-info
-rw-r----- 1 root root 0 Jan  1  1970 hex
-rw-r----- 1 root root 0 Jan  1  1970 hash-ptr
-rw-r----- 1 root root 0 Jan  1  1970 function-trace
-rw-r----- 1 root root 0 Jan  1  1970 function-fork
-rw-r----- 1 root root 0 Jan  1  1970 fields
-rw-r----- 1 root root 0 Jan  1  1970 event-fork
-rw-r----- 1 root root 0 Jan  1  1970 display-graph
-rw-r----- 1 root root 0 Jan  1  1970 disable_on_free
-rw-r----- 1 root root 0 Jan  1  1970 context-info
-rw-r----- 1 root root 0 Jan  1  1970 block
-rw-r----- 1 root root 0 Jan  1  1970 bin
-rw-r----- 1 root root 0 Jan  1  1970 annotate
```

```
-rw-r----- 1 root root 0 Jan  1  1970 test_nop_refuse
-rw-r----- 1 root root 0 Jan  1  1970 test_nop_accept
-rw-r----- 1 root root 0 Jan  1  1970 sleep-time
-rw-r----- 1 root root 0 Jan  1  1970 graph-time
-rw-r----- 1 root root 0 Jan  1  1970 func_stack_trace
-rw-r----- 1 root root 0 Jan  1  1970 func-no-repeats
-rw-r----- 1 root root 0 Jan  1  1970 funcgraph-tail
-rw-r----- 1 root root 0 Jan  1  1970 funcgraph-overrun
-rw-r----- 1 root root 0 Jan  1  1970 funcgraph-overhead
-rw-r----- 1 root root 0 Jan  1  1970 funcgraph-irqs
-rw-r----- 1 root root 0 Jan  1  1970 funcgraph-duration
-rw-r----- 1 root root 0 Jan  1  1970 funcgraph-cpu
-rw-r----- 1 root root 0 Jan  1  1970 blk_classic
-rw-r----- 1 root root 0 Jan  1  1970 blk_cgroup
-rw-r----- 1 root root 0 Jan  1  1970 blk_cgname
-rw-r----- 1 root root 0 Feb 28 15:17 funcgraph-proc
-rw-r----- 1 root root 0 Feb 28 15:17 funcgraph-abstime
```

8. Disabling interrupts info in function graph

The function graph tracer also traces the interrupts.

Sometimes they clutter the trace

You can tell the function graph tracer to disable tracing of interrupts by writing 0 in 'options/funcgraph-irqs' file.

```
echo 0 > options/funcgraph-irqs
```

After I disable interrupts, there is no tracing information related to interrupts (e.g. smp_apic_timer_interrupt).

```
cat trace | grep -B 5 -A 5 "======>" | head -n 20
cat trace | grep smp_apic_timer_interrupt
cat trace | grep smp_apic_timer_interrupt | wc -l
```

9. Who calls this kernel function

With ftrace, you can find out who is calling a particular kernel function

"function" tracer contains an option which creates backtrace for each function called by tracer: func_stack_trace

At this function, give me stack dump. Opposite of call graph

> Warning: Can cause live lock if all functions are traced !!
> Must use set_ftrace_filter Before enabling

Note: You can use this feature only when the tracer is "function".

Example1

```
echo vfs_read > set_ftrace_filter
echo function > current_tracer
echo 1 > options/func_stack_trace
```

Example2

```
echo nop > current_tracer
echo schedule > set_ftrace_filter
cat set_ftrace_filter # double check
echo 1 > options/func_stack_trace
echo function > current_tracer
sleep 1
echo 0 > tracing_on
echo 0 > options/func_stack_trace
cat trace
```

10. trace_marker

Seeing what happens inside the kernel gives the user a better understanding of how their system works

But sometimes there needs to be coordination between what is happening in user space and what is happening inside the kernel

Let's say you are doing something in the user space and you want to synchronize with the tracing, trace_marker file was created

It provides a way to write into the Ftrace ring buffer from user space.

```
echo nop > current_tracer
echo 'hello world!' > trace_marker
cat trace
```

Whatever you wrote is in the trace buffer

```
root@RPI-3:~/ktrace_practice# gcc userprog1.c -o userprog1
root@RPI-3:/sys/kernel/tracing# cat tracing_on
0
root@RPI-3:/sys/kernel/tracing# cat current_tracer
function
root@RPI-3:/sys/kernel/tracing#
root@RPI-3:~/ktrace_practice# ./userprog1
root@RPI-3:/sys/kernel/tracing# cat trace | grep "Before" -A 5
            prac-16269   [001] ..... 435630.849122: tracing_mark_write: Before printf
          <idle>-0       [003] d.h1. 435630.849123: add_interrupt_randomness <-handle_irq_event
          <idle>-0       [003] d.h1. 435630.849125: fast_mix <-add_interrupt_randomness
          <idle>-0       [003] d.h1. 435630.849126: note_interrupt <-handle_irq_event
          <idle>-0       [003] d.h1. 435630.849128: _raw_spin_lock <-handle_irq_event
          <idle>-0       [003] d.h2. 435630.849130: armctrl_unmask_irq <-handle_level_irq
root@RPI-3:/sys/kernel/tracing#
root@RPI-3:/sys/kernel/tracing# cat trace | grep "Before" -B 5
            prac-16269   [001] ..... 435630.849117: rw_verify_area <-vfs_write
          <idle>-0       [003] d.h2. 435630.849117: _raw_spin_unlock <-handle_irq_event
            prac-16269   [001] ..... 435630.849119: security_file_permission <-rw_verify_area
          <idle>-0       [003] d.h1. 435630.849119: __handle_irq_event_percpu <-handle_irq_event
          <idle>-0       [003] d.h1. 435630.849121: fiq_irq_handler <-__handle_irq_event_percpu
            prac-16269   [001] ..... 435630.849122: tracing_mark_write: Before printf
root@RPI-3:/sys/kernel/tracing#
root@RPI-3:/sys/kernel/tracing#
root@RPI-3:/sys/kernel/tracing# cat trace | grep "Before" -C 5
            prac-16269   [001] ..... 435630.849117: rw_verify_area <-vfs_write
          <idle>-0       [003] d.h2. 435630.849117: _raw_spin_unlock <-handle_irq_event
            prac-16269   [001] ..... 435630.849119: security_file_permission <-rw_verify_area
          <idle>-0       [003] d.h1. 435630.849119: __handle_irq_event_percpu <-handle_irq_event
          <idle>-0       [003] d.h1. 435630.849121: fiq_irq_handler <-__handle_irq_event_percpu
            prac-16269   [001] ..... 435630.849122: tracing_mark_write: Before printf
```

```
<idle>-0       [003] d.h1. 435630.849123: add_interrupt_randomness <-handle_irq_event
<idle>-0       [003] d.h1. 435630.849125: fast_mix <-add_interrupt_randomness
<idle>-0       [003] d.h1. 435630.849126: note_interrupt <-handle_irq_event
<idle>-0       [003] d.h1. 435630.849128: _raw_spin_lock <-handle_irq_event
<idle>-0       [003] d.h2. 435630.849130: armctrl_unmask_irq <-handle_level_irq
```

userprog1.c

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdarg.h>
#include <unistd.h>
#include <stdlib.h>

static int marker_fd = -1;

static void setup_trace_marker(void)
{
	marker_fd = open("/sys/kernel/tracing/trace_marker", O_WRONLY);
}

static void write_trace_marker(const char *fmt, ...)
{
	char buf[2048];
	va_list ap;
	int n;

	if (marker_fd < 0)
		return;

	va_start(ap, fmt);
	n = vsnprintf(buf, 2048, fmt, ap);
	va_end(ap);
	write(marker_fd, buf, n);

}

int main()
{
	int tracing_on_fd;
	int ret;

	tracing_on_fd = open("/sys/kernel/debug/tracing/tracing_on", O_WRONLY);
	if (tracing_on_fd < 0) {
```

```
        perror("error:");
        exit(-1);
    }
    setup_trace_marker();
    ret = write(tracing_on_fd, "1", 1);
    if (ret < 0) {
        perror("error:");
        exit(-1);
    }
    write_trace_marker("Before printf");
    printf("Hello Printf\n");
    write_trace_marker("After printf");
    ret = write(tracing_on_fd, "0", 1);
    if (ret < 0) {
        perror("error:");
        exit(-1);
    }
    close(marker_fd);
    close(tracing_on_fd);
    return 0;
}
```

Github : https://github.com/kd1995-max/Ftrace-practice/blob/main/section-2/10/userprog.c

11. Kernel function profiling

From Wikipedia, function profiling provides you the following :

Space or Time Complexity of a Program
Usage of Particular Instruction
Frequency and duration of function calls

ftrace can provide you the following function profiling information using 'function_profiling_enabled' file.

Frequency of function calls (When the tracer is 'function')
Duration spent in each function calls (When the tracer is 'function_graph')

Steps for Function profiling:

1. Enable the function tracer: 'function'/'function_graph'
```
echo 'function' > current_tracer
```
2. Enable the function profiling:
```
echo '1' > function_profile_enabled
```

Observing Results

To observe the results look into '/sys/kernel/tracing/trace_stat' folder.

You will observe a lot of file with the following name: function<number> (e.g. function0, function1,...), where number represents the core on which the function is executing.

12. Finding out maximum time interrupts are disabled

When interrupts are disabled, the CPU can not react to any other external event

timer interrupts
mouse interrupts

This prevents the timer interrupt from triggering or the mouse interrupt from letting the kernel know of a new mouse event.

The result is a latency with the reaction time.

irqsoff is a tracer which tracks the times in the kernel when interrupts are disabled.

Whenever a new maximum latency is found, the tracer saves the trace leading up to that latency point.

Each time a new maximum latency is reached, the old saved trace is discarded and the new trace is saved.

By default this tracer may not be enabled in your kernel, you need to re-compile my linux kernel with the following kernel configuration: "CONFIG_IRQSOFF_TRACER"

```
cat available_tracers
echo irqsoff > current_tracer
echo 1 > tracing_on
cat trace
cat tracing_max_latency
```

To reset the maximum latency, write '0' in 'tracing_max_latency' file.

```
echo 0 > tracing_max_latency
```

13. Finding latencies at kernel startup

Kernel provides many command-line parameters for ftrace

1. **ftrace = [tracer] :** This will set and start the tracer as early as possible in boot
2. **ftrace_filter=[function_list]:** Function-list is a comma-separated list of functions. Traces only the functions present in the filter at boot.
3. **ftrace_graph_filter=[function_list]:** Limits the top level callers functions traced by function graph tracer
4. **ftrace_notrace, ftrace_graph_notrace=[function_list]:** Opposite of ftrace_filter, and ftrace_graph_filter. Will not trace the functions listed in this parameter.
5. **ftrace_graph_max_depth=<uint>:** Used along with function graph tracer. This is max depth it will trace into the function
6. **tracing_thresh=<uint>:** Will trace only the functions taking longer than passed values.

Example1

```
ftrace=function_graph ftrace_graph_max_depth=1
```

This tells the kernel to start 'function_graph' tracer on start and maximum depth of tracing into function is 1.

Example2

```
ftrace=function_graph tracing_thresh=200
```

this traces all functions taking longer than 200 microseconds (.2 ms). You can use any duration threshhold you want.

Note: the tracer starts sometime during initialization, and you only get timings after it starts

14. Find deepest kernel stack

Kernel has a fixed stack size.

The default stack size for process running in kernel is 8K.

Call a function in recursion without any break, you will observe system will freeze within seconds

A kernel developer should be careful with what they are allocating on the stack

If he adds a lot to the kernel stack, stack overflow can happen and finally system panic.

Ftrace helps you to find the routine with the deepest kernel stack.

Turning it on:
You can turn it on at boot time or at runtime.

Boot time:
use the following on the kernel command line:
stacktrace

Run Time:

```
echo 1 >/proc/sys/kernel/stack_tracer_enabled
```

Wait for few minutes to get a better information.

the stack tracer will continue operating until you turn it off, which can be done with:

```
echo 0 >/proc/sys/kernel/stack_tracer_enabled
```

Files to look to get stack information:

1. **stack_max_size:** Displays the maximum size it has encountered
2. **stack_trace:** Displays the backtrace of the largest stack trace encountered after activating stack tracer, it also shows the breakdown of the stack sizes used by each function
3. **stack_trace_filter:** Limits what functions stack tracer will check, it is similar to "set_ftrace_filter"

To reset the maximum, echo “0” into the stack_max_size file.

```
echo 0 > stack_max_size
```

15. Capturing Ftrace to oops when kernel panic

You can capture the function calls leading up to a panic using ftrace

Two Ways:

1. Boot time:

Add the following to kernel command line: ftrace_dump_on_oops

2. Run time:

```
echo 1 > /proc/sys/kernel/ftrace_dump_on_oops
```

will enable Ftrace to dump to the console the entire trace buffer in ASCII format on oops or panic

Having the console output to a serial log makes debugging crashes much easier.

You can now trace back the events that led up to the crash.

Dumping to the console may take a long time since the default Ftrace ring buffer is over a megabyte per CPU.

To shrink the size of the ring buffer, write the number of kilobytes you want the ring buffer to be to buffer_size_kb.

```
[tracing]# echo 50 > buffer_size_kb
```

The above will shrink the Ftrace ring buffer down to 50 kilobytes per CPU

16. snapshot

Reading the trace buffer can cause events to be lost

Displays the snapshot buffer
Allows the user to take a snapshot of the current running trace without stopping tracing
Ftrace swaps the current buffer with a spare buffer, and tracing continues in the new current (=previous spare) buffer.

```
echo 1 > snapshot
cat snapshot
```

Required Kernel Configuration: CONFIG_TRACER_SNAPSHOT

SECTION - 3
1. Tracing Events

Why function tracing is not enough

Function tracing (ftrace) answers only one question:

> *Which function was called and when?*

It does **not** tell you:

- What data the function was working on
- What arguments were passed
- What decisions were made inside the function

For real kernel debugging, this is often **not sufficient**.

Example problem:
You see `schedule()` being called, but:

- Why was the task scheduled out?
- What was the previous task?
- What was the next task?
- What state triggered the switch?

Function tracing cannot answer this.

What tracepoints really are (core idea)

A **tracepoint** is a **predefined, explicit instrumentation hook** placed by kernel developers at *important logic points* in the kernel.

Think of a tracepoint as:

> "At this exact place in the code, if someone is interested, give them structured internal data."

It is **not dynamic guessing**, unlike kprobes.
It is **intentional design-time instrumentation**.

Where tracepoints live in code

Tracepoints are placed at:

- Scheduler decisions
- IRQ entry/exit
- Power state changes
- Network packet receive/transmit
- Lock contention
- GPIO operations

These are **semantic points**, not random lines of code.

Example (scheduler):

```
trace_sched_switch(prev, next);
```

This line exists because kernel developers decided:

> "Task switching is important enough to expose."

Tracepoints vs ftrace (key distinction)

ftrace

- Hooks **function entry/exit**
- Sees only function name and timing
- No structured data
- No semantic meaning

Tracepoints

- Hook **specific kernel events**
- Carry **explicit data fields**
- Capture **local variables**
- Semantically meaningful

In short:

> ftrace shows *control flow*
> tracepoints show *state and intent*

How tracepoints work internally

A tracepoint has four logical components.

1. Tracepoint definition (TRACE_EVENT)

This is written by kernel developers using `TRACE_EVENT`.

This macro:

- Defines the event name
- Defines what data is captured
- Defines the data layout

Example (conceptual):

```
TRACE_EVENT(sched_switch,
    TP_PROTO(struct task_struct *prev, struct task_struct *next),
    TP_ARGS(prev, next),
    ...
);
```

This **declares**:

- What arguments exist
- What fields will be recorded
- What types they have

2. Tracepoint call site

Inside kernel code:

trace_sched_switch(prev, next);

This is just a normal function-like call.

Important detail:

- If tracing is **disabled**, this call is almost free
- It compiles down to a static jump / no-op

This is why tracepoints are safe for production kernels.

3. Runtime enable/disable mechanism

Each tracepoint can be:

- Completely OFF
- Enabled dynamically at runtime

When OFF:

- No buffer writes
- No formatting
- No callbacks
- Minimal branch cost

When ON:

- Data is captured
- Written into the **ring buffer**
- Timestamped
- Stored per-CPU

4. Event formatting and user-space visibility

Each TRACE_EVENT also generates:

- A binary layout (for speed)
- A human-readable format (for tools)

This allows tools like:

- tracefs
- perf
- ftrace event interface

To display output such as:

```
prev_comm=bash prev_pid=1234 next_comm=sshd next_pid=4321
```

Why tracepoints can see local variables

This is critical.

Because tracepoints are:

- Written **inside** the function
- At a point where variables already exist

They can capture:

- Function arguments
- Local variables
- Derived state

Example:

```
int ret = calculate_load(cpu);
trace_cpu_load(cpu, ret);
```

This is **impossible** with function tracing.

2. Tracing Events (available_events)

The list of events can be viewed by running the command

```
cd /sys/kernel/tracing
cat available_events
```

A long list will be printed out in the console.

This is a bit inconvenient. We can print out a more structured listed using the command:

```
ls /sys/kernel/tracing/events
```

Events are grouped by sub system

- **sched -** schedule events
- **irq -** interrupt events
- **net -** networking events
- **syscalls -** system call events (all system calls and their parameters)
- **module -** module loading, unloading, freeing etc
- **kvm -** events for the KVM guests
- **exceptions -** page faults
- **cgroup -** changes to cgroups
- And many more

Each sub system has several events

For example: The "sched" system

- sched_switch
- sched_waking
- sched_process_fork
- sched_process_exit
- sched_process_exec
- sched_migrate_task
- etc

```
ls events/sched
```

How to enable events?

Using enable file

```
echo nop > current_tracer
echo 1 > events/sched/sched_waking/enable
echo 1 > events/sched/sched_switch/enable
cat trace
```

Enabling Group of Events

If you want to enable group of events, for example enable all sched events

```
echo nop > current_tracer
echo 1 > events/sched/enable
cat trace
echo 1 > events/irq/enable
echo 1 > events/timer/enable
```

Enabling All Events

```
echo 1 > events/enable
cat trace
```

You can do the other way as well, enable all the events and disable the events that you don't want by writing 0 in the enable file

Enable file

When reading one of these enable files, there are four results:

0 - all events this file affects are disabled

1 - all events this file affects are enabled

X - there is a mixture of events enabled and disabled

? - this file does not affect any event

3. Format file

Each trace event has a **format file** located at:

```
/sys/kernel/tracing/events/<system>/<event>/format
```

This file **describes exactly how one trace record is laid out in memory** and how it will be printed.

id

- `ID` is a **unique numeric identifier** for the trace event.
- It is used internally by the **ring buffer** to identify which event type a record belongs to.
- For `sched_switch`, the ID is `323`.

format section

The `format:` section describes **all fields stored in the trace buffer** for this event.

Each field is described as:

field:<type> <name>; offset:<N>; size:<N>;

- **offset** → byte offset inside the trace record
- **size** → size of the field in bytes

This layout is **binary**, not text.

Common fields (present in all events)

Fields prefixed with `common_` exist in **every trace event**.

1. **common_type**
2. **common_flags**
3. **common_preempt_count**
4. **common_pid**

Meaning:

- **common_type**
  - Event type ID (same as `ID`)
- **common_flags**
  - IRQ state (interrupts enabled/disabled)

- **common_preempt_count**
    - Preemption state and context info (process / softirq / hardirq / NMI)
- **common_pid**
    - PID of the task that triggered the event

These fields are added automatically by the tracing framework.

Event-specific fields

After common fields, the remaining fields come directly from the `TRACE_EVENT` definition.

For `sched_switch`:

prev_comm

prev_pid

prev_prio

prev_state

next_comm

next_pid

next_prio

print fmt

The `print fmt:` line defines **how the binary data is converted to text**.

- Uses the recorded fields (`REC->field_name`)
- Applies helpers like `__print_flags()` for decoding bitmasks
- This is what tools like `trace-cmd` or `cat trace` display

Binary data → formatted human-readable output.

```
root@RPI-3:/sys/kernel/tracing# cat events/sched/sched_switch/format
name: sched_switch
ID: 249
format:
        field:unsigned short common_type;       offset:0;       size:2; signed:0;
        field:unsigned char common_flags;       offset:2;       size:1; signed:0;
        field:unsigned char common_preempt_count;       offset:3;       size:1; signed:0;
        field:int common_pid;   offset:4;       size:4; signed:1;

        field:char prev_comm[16];       offset:8;       size:16;        signed:0;
        field:pid_t prev_pid;   offset:24;      size:4; signed:1;
        field:int prev_prio;    offset:28;      size:4; signed:1;
        field:long prev_state;  offset:32;      size:8; signed:1;
        field:char next_comm[16];       offset:40;      size:16;        signed:0;
        field:pid_t next_pid;   offset:56;      size:4; signed:1;
        field:int next_prio;    offset:60;      size:4; signed:1;

print fmt: "prev_comm=%s prev_pid=%d prev_prio=%d prev_state=%s%s ==> next_comm=%s next_pid=%d
next_prio=%d", REC->prev_comm, REC->prev_pid, REC->prev_prio, (REC->prev_state & ((((0x00000000 |
0x00000001 | 0x00000002 | 0x00000004 | 0x00000008 | 0x00000010 | 0x00000020 | 0x00000040) + 1) << 1)
- 1)) ? __print_flags(REC->prev_state & ((((0x00000000 | 0x00000001 | 0x00000002 | 0x00000004 |
0x00000008 | 0x00000010 | 0x00000020 | 0x00000040) + 1) << 1) - 1), "|", { 0x00000001, "S" }, {
0x00000002, "D" }, { 0x00000004, "T" }, { 0x00000008, "t" }, { 0x00000010, "X" }, { 0x00000020, "Z"
}, { 0x00000040, "P" }, { 0x00000080, "I" }) : "R", REC->prev_state & (((0x00000000 | 0x00000001 |
0x00000002 | 0x00000004 | 0x00000008 | 0x00000010 | 0x00000020 | 0x00000040) + 1) << 1) ? "+" : "",
REC->next_comm, REC->next_pid, REC->next_prio
root@RPI-3:/sys/kernel/tracing#
```

4. Filtering Events

Filtering events is done by writing a filter to the filter file

Filters can be set on each parameter listed in the format file, whether it is a numeric or string

How it works?

As soon as an event is logged into the trace buffer, its fields are checked against the filter expression associated with that event type

An event with field values that 'match' the filter will appear in the trace output

An event whose values don't match will be discarded

An event with no filter associated with it matches everything, and is the default when no filter has been set for an event.

Filter Expressions

A filter expression consists of one or more 'predicates' that can be combined using the logical operators '&&' and '||'

Predicate: a clause that compares the value of a field contained within a logged event with a constant value and returns either 0 or 1

Return value depends on whether the field value matched (1) or didn't match (0)

Syntax of predicate: field-name relational-operator value

Parentheses can be used to provide arbitrary logical groupings and double-quotes can be used to prevent the shell from interpreting operators as shell metacharacters.

The field-names available for use in filters can be found in the 'format' files for trace events

The relational-operators depend on the type of the field being tested:

Numeric Fields: ==, !=, <, <=, >, >=, &

String Fields: ==, !=, ~

The glob (~) accepts a wild card character (*,?) and character classes ([)

Example 1:

```
echo 'prev_comm == "bash" && prev_state & 0x02' > events/sched/sched_switch/filter
echo 1 > events/sched/sched_switch/enable
echo > trace
cat trace
```

0x02 means D state. It's blocked on something

Clearing filters

To clear the filter for an event, write a ‘0’ to the event’s filter file.

To clear the filters for all events in a subsystem, write a ‘0’ to the subsystem’s filter file.

5. Setting filters - More examples

A filter for an individual event is set by writing a filter expression to the 'filter' file for the given event

```
cd /sys/kernel/debug/tracing/events/sched/sched_switch
echo "common_pid == 0" > filter
```

A slightly more involved example:

- signal_generate
- signal_deliver

```
cd /sys/kernel/debug/tracing/events/signal/signal_generate
echo "((sig >= 10 && sig < 15) || sig == 17) && comm != bash" > filter
```

If there is an error in the expression, you'll get an 'Invalid argument' error when setting it

the erroneous string along with an error message can be seen by looking at the filter

```
cd /sys/kernel/debug/tracing/events/signal/signal_generate
echo "((sig >= 10 && sig < 15) || dsig == 17) && comm != bash" > filter
cat filter
```

Github: https://github.com/kd1995-max/Ftrace-practice/tree/main/section-⅗

6. Subsystem filters

Filters for every event in a subsystem can be set or cleared as a group by writing a filter expression into the filter file at the root of the subsystem

Note:

The filter for that event retains its previous setting

if a filter for any event within the subsystem lacks a field specified in the subsystem filter,

if the filter can't be applied for any other reason

Only filters that reference just the common fields can be guaranteed to propagate successfully to all events

Example 1

```
cd /sys/kernel/debug/tracing/events/signal
echo 'sig==14' > filter
cat signal_deliver/filter
cat signal_generate/filter
```

Example 2

```
cd /sys/kernel/debug/tracing/events/sched
echo common_pid == 0 > filter
cat sched_switch/filter
 common_pid == 0
cat sched_wakeup/filter
 common_pid == 0
```

Example 3

```
# cd /sys/kernel/debug/tracing/events/sched
# echo prev_pid == 0 > filter
# cat sched_switch/filter
prev_pid == 0
# cat sched_wakeup/filter
common_pid == 0
```

7. Event Triggers

Make an event do something special

- Turn off tracing
- Turn on tracing
- Take a "snapshot"
- Produce a stack dump
- Enable another event
- Disable another event

What does a trigger do?

Trigger actually allows you to make an action happens on a trace event

Whenever a trace event with attached triggers is invoked, the set of trigger commands associated with that event is invoked.

Syntax:

Triggers are added by echoing the command to the 'trigger' file:

```
echo 'command[:count] [if filter]' > trigger
```

Triggers are removed by echoing the same command but starting with '!' to the 'trigger' file

```
echo '!command[:count] [if filter]' > trigger
```

You can ignore [if filter] part while removing triggers

Following commands are supported:

enable_event/disable_event:

These commands can enable or disable another trace event whenever the triggering event is hit.

Format:

```
enable_event:<system>:<event>[:count]
disable_event:<system>:<event>[:count]

echo 'enable_event:kmem:kmalloc:1' > \
    /sys/kernel/debug/tracing/events/syscalls/sys_enter_read/trigger

echo 'disable_event:kmem:kmalloc' > \
    /sys/kernel/debug/tracing/events/syscalls/sys_exit_read/trigger
```

sys_enter_* used to get function arguments and sys_exit_* to get return value

To remove the above commands:

```
echo '!enable_event:kmem:kmalloc:1' > \
    /sys/kernel/debug/tracing/events/syscalls/sys_enter_read/trigger
echo '!disable_event:kmem:kmalloc' > \
    /sys/kernel/debug/tracing/events/syscalls/sys_exit_read/trigger
```

stacktrace:

This command dumps a stacktrace in the trace buffer whenever the triggering event occurs.

Format:

stacktrace[:count]

```
echo 'stacktrace' > \
      /sys/kernel/debug/tracing/events/kmem/kmalloc/trigger
echo 'stacktrace:5 if bytes_req >= 65536' > \
      /sys/kernel/debug/tracing/events/kmem/kmalloc/trigger
```

To remove the above commands:

```
echo '!stacktrace' > \
      /sys/kernel/debug/tracing/events/kmem/kmalloc/trigger
echo '!stacktrace:5 if bytes_req >= 65536' > \
      /sys/kernel/debug/tracing/events/kmem/kmalloc/trigger
```

The latter one can also be removed without adding the filter

```
echo '!stacktrace:5' > \
      /sys/kernel/debug/tracing/events/kmem/kmalloc/trigger
```

Other commands:

snapshot:

This command causes a snapshot to be triggered whenever the triggering event occurs.

traceon/traceoff:

These commands turn tracing on and off when the specified events are hit

hist:

Histogram specific

8. Histogram

Hist triggers is a new tracing feature that recently landed in Linux 4.7

It can be used to aggregate trace event data into histograms.

A histogram trigger command is an event trigger command that aggregates event hits into a hash table keyed on one or more trace event format fields (or stacktrace)

Format:

```
hist:keys=<field1[,field2,...]>[:values=<field1[,field2,...]>]
  [:sort=<field1[,field2,...]>][:size=#entries][:pause][:continue]
  [:clear][:name=histname1] [if <filter>]
```

How it works

When a matching event is hit, an entry is added to a hash table using the key(s) and value(s) named.

Keys and values correspond to fields in the event's format description.

Values must correspond to numeric fields - on an event hit, the value(s) will be added to a sum kept for that field.

If 'values' isn't specified, an implicit 'hitcount' value (simply a count of event hits) will be automatically created and used as the only value.

By default, the size of the hash table is 2048 entries. The 'size' parameter can be used to specify more or fewer than that.

Example1:

common_pid.execname display a common_pid as a program name

```
echo 'hist:key=common_pid.execname' > \
      /sys/kernel/debug/tracing/events/raw_syscalls/sys_enter/trigger
```

Wait for few seconds

```
cat /sys/kernel/debug/tracing/events/raw_syscalls/sys_enter/hist

echo '!hist:key=common_pid.execname' > \
      /sys/kernel/debug/tracing/events/raw_syscalls/sys_enter/trigger
```

Example2:

```
echo 'hist:key=common_pid.execname,ret' > \
      /sys/kernel/debug/tracing/events/syscalls/sys_exit_read/trigger


cat /sys/kernel/debug/tracing/events/syscalls/sys_exit_read/hist
 echo '!hist:key=common_pid.execname,ret' > \
      /sys/kernel/debug/tracing/events/syscalls/sys_exit_read/trigger
```

Example3

```
cd /sys/kernel/debug/tracing/events/kmem/kmalloc
 cat format
echo 'hist:key=call_site:values=bytes_req' > trigger
cat trigger
 cat hist
 echo '!hist:key=call_site:values=bytes_req' > trigger
```

Default sort order is 'ascending'. To sort in the opposite direction, append .descending' to the sort key.

```
echo 'hist:key=call_site:values=bytes_req:sort=bytes_req.descending' > trigger
cat hist
echo '!hist:key=call_site:values=bytes_req:sort=bytes_req.descending' > trigger
```

9. Event Tracing via the 'set_event' interface

The events which are available for tracing can be found in the file: `/sys/kernel/debug/tracing/available_events`

To enable a particular event, such as 'sched_wakeup', simply echo it to `/sys/kernel/debug/tracing/set_event.`

```
echo sched_wakeup >> /sys/kernel/debug/tracing/set_event
```

Note: '>>' is necessary, otherwise it will firstly disable all the events.

To disable an event, echo the event name to the set_event file prefixed with an exclamation point

```
echo '!sched_wakeup' >> /sys/kernel/debug/tracing/set_event
```

To disable all events, echo an empty line to the set_event file:

```
echo > /sys/kernel/debug/tracing/set_event
```

To enable all events, echo *:* or *: to the set_event file:

```
echo *:* > /sys/kernel/debug/tracing/set_event
```

The events are organized into subsystems, such as ext4, irq, sched, etc., and a full event name looks like this: <subsystem>:<event>.

The subsystem name is optional, but it is displayed in the available_events file.

All of the events in a subsystem can be specified via the syntax <subsystem>:*; for example, to enable all irq events, you can use the command:

```
echo 'irq:*' > /sys/kernel/debug/tracing/set_event
```

10. PID filtering

set_event_pid file traces events from task with a PID listed in this file

```
echo $$ > set_event_pid
```

Github: https://github.com/kd1995-max/Ftrace-practice/blob/main/section-3/10/userprog.c

11. Tracing Event at Boot

Use the following kernel command line to start tracing events on boot

trace_event=[event-list]

event-list is a comma separated list of events.

Eg. Boot time tracing

trace_event=initcall:*

Step-by-step (works for most Linux setups that use **GRUB**). I'll keep it direct.

1) Decide what you want to trace

Example:

trace_event=initcall:*

2) Add it to the kernel command line

If your system uses GRUB

1. Open GRUB default config:

```
sudo nano /etc/default/grub
```

2. Find this line:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
```

3. Add the tracing parameter inside the quotes:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash trace_event=initcall:*"
```

4. Save and exit.

3) Update GRUB

Run:

```
sudo update-grub
```

4) Reboot

```
sudo reboot
```

5) Read the boot trace after reboot

1. Go to tracefs:

```
cd /sys/kernel/tracing
```

2. Read trace:

```
sudo cat trace
```